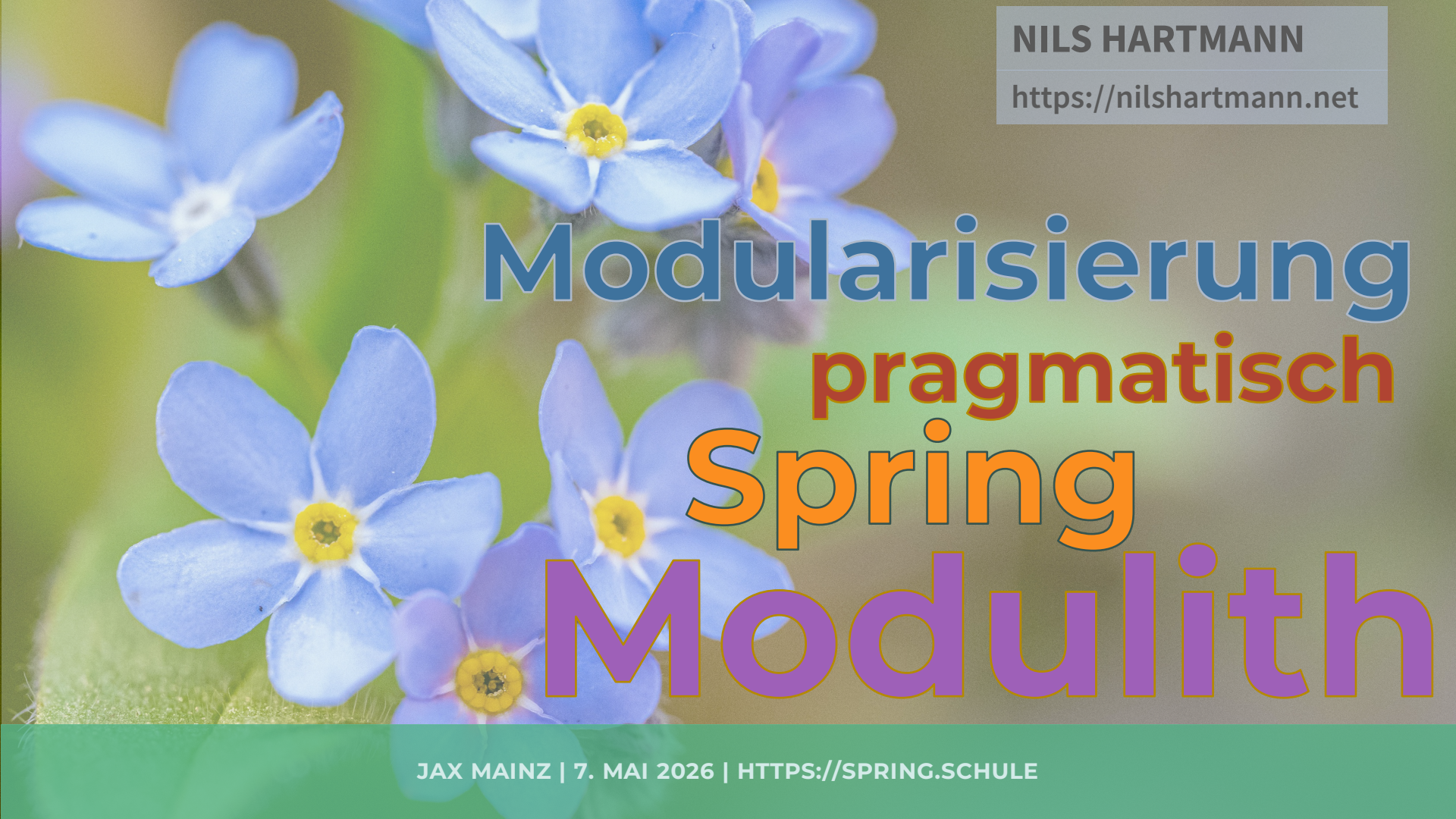




NILS HARTMANN

<https://nilshartmann.net>

Modularisierung pragmatisch Spring Modulith

JAX MAINZ | 7. MAI 2026 | [HTTPS://SPRING.SCHULE](https://spring.schule)

NILS HARTMANN

nils@nilshartmann.net

Freiberuflicher Software-Entwickler, –Architekt, Coach, Trainer

Java, Spring, GraphQL, React, TypeScript



<https://react.schule/video-kurs-react>



<https://reactbuch.de>

[HTTPS://NILSHARTMANN.NET](https://nilshartmann.net)



Q Search

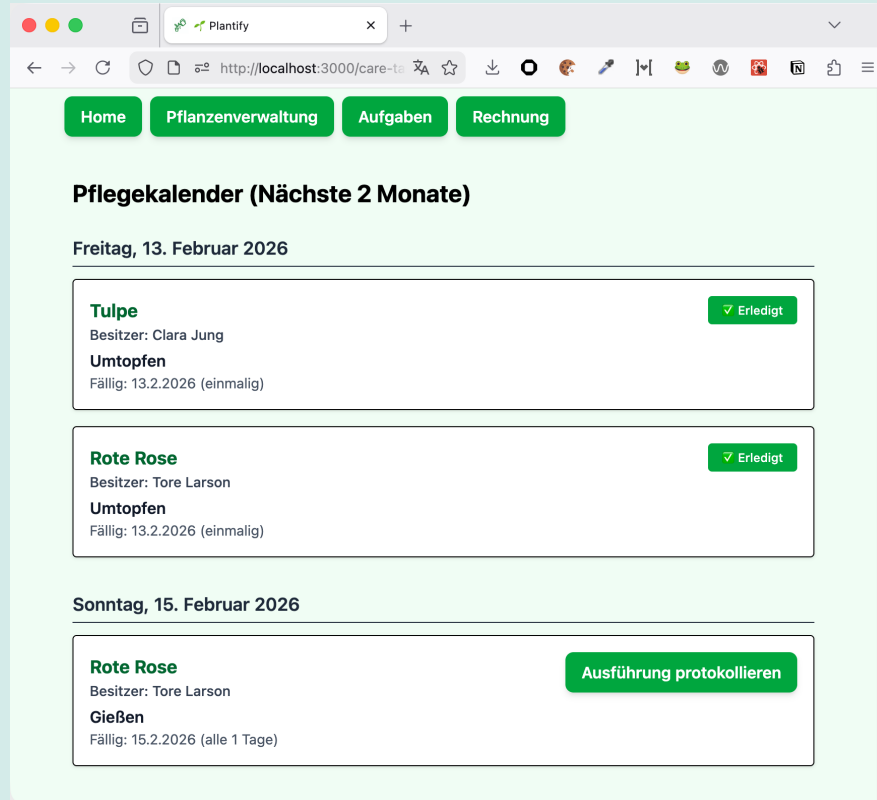
⌘ + k



Fundamentals

Spring Modulith supports developers implementing logical **modules** in Spring Boot applications. It allows them to apply structural **validation, document** the module arrangement, run integration tests for individual modules, observe the modules' interaction at runtime, and generally implement module interaction in a **loosely coupled** way.

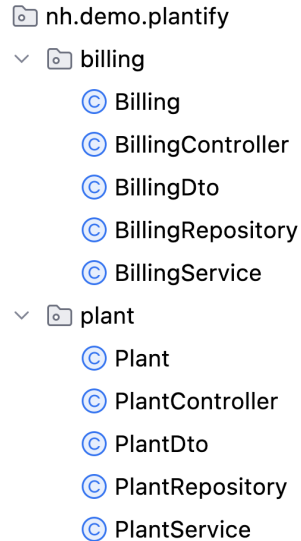
<https://docs.spring.io/spring-modulith/reference/fundamentals.html>



BEISPIEL

- Gliederung der Anwendung in fachliche Packages

- Gliederung der Anwendung in fachliche Packages



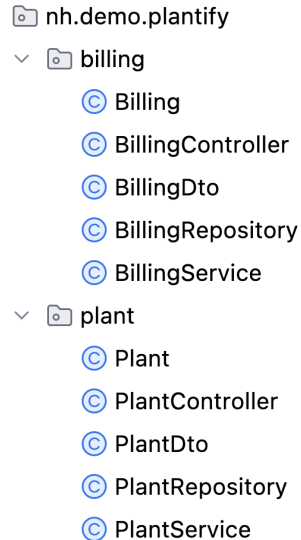
```
graph TD; nh_demo_plantify[nh.demo.plantify] --> billing[billing]; nh_demo_plantify --> plant[plant]; billing --> Billing[Billing]; billing --> BillingController[BillingController]; billing --> BillingDto[BillingDto]; billing --> BillingRepository[BillingRepository]; billing --> BillingService[BillingService]; plant --> Plant[Plant]; plant --> PlantController[PlantController]; plant --> PlantDto[PlantDto]; plant --> PlantRepository[PlantRepository]; plant --> PlantService[PlantService];
```

nh.demo.plantify

- ▼ billing
 - Ⓢ Billing
 - Ⓢ BillingController
 - Ⓢ BillingDto
 - Ⓢ BillingRepository
 - Ⓢ BillingService
- ▼ plant
 - Ⓢ Plant
 - Ⓢ PlantController
 - Ⓢ PlantDto
 - Ⓢ PlantRepository
 - Ⓢ PlantService

- Fachlichkeit ist in **einem** Package konzentriert

- Gliederung der Anwendung in fachliche Packages



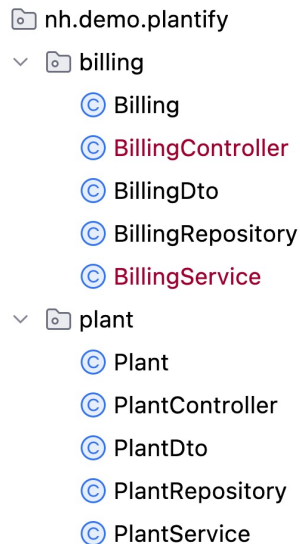
```
graph TD; nh_demo_plantify[nh.demo.plantify] --> billing[billing]; nh_demo_plantify --> plant[plant]; billing --> Billing[Billing]; billing --> BillingController[BillingController]; billing --> BillingDto[BillingDto]; billing --> BillingRepository[BillingRepository]; billing --> BillingService[BillingService]; plant --> Plant[Plant]; plant --> PlantController[PlantController]; plant --> PlantDto[PlantDto]; plant --> PlantRepository[PlantRepository]; plant --> PlantService[PlantService];
```

nh.demo.plantify

- ▼ billing
 - ⊙ Billing
 - ⊙ BillingController
 - ⊙ BillingDto
 - ⊙ BillingRepository
 - ⊙ BillingService
- ▼ plant
 - ⊙ Plant
 - ⊙ PlantController
 - ⊙ PlantDto
 - ⊙ PlantRepository
 - ⊙ PlantService

- Fachlichkeit ist in **einem** Package konzentriert
- Vertikaler „**Slice**“, enthält alle Schichten

- Gliederung der Anwendung in fachliche Packages



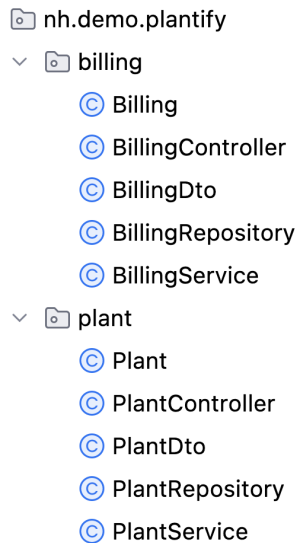
```
graph TD; nh_demo_plantify[nh.demo.plantify] --> billing[billing]; nh_demo_plantify --> plant[plant]; billing --> Billing[Billing]; billing --> BillingController[BillingController]; billing --> BillingDto[BillingDto]; billing --> BillingRepository[BillingRepository]; billing --> BillingService[BillingService]; plant --> Plant[Plant]; plant --> PlantController[PlantController]; plant --> PlantDto[PlantDto]; plant --> PlantRepository[PlantRepository]; plant --> PlantService[PlantService];
```

nh.demo.plantify

- ▼ billing
 - © Billing
 - © BillingController
 - © BillingDto
 - © BillingRepository
 - © BillingService
- ▼ plant
 - © Plant
 - © PlantController
 - © PlantDto
 - © PlantRepository
 - © PlantService

- Fachlichkeit ist in **einem** Package konzentriert
- Vertikaler „**Slice**“, enthält alle Schichten
- **Änderungen** müssen nur an **einer Stelle** gemacht werden

- Gliederung der Anwendung in fachliche Packages



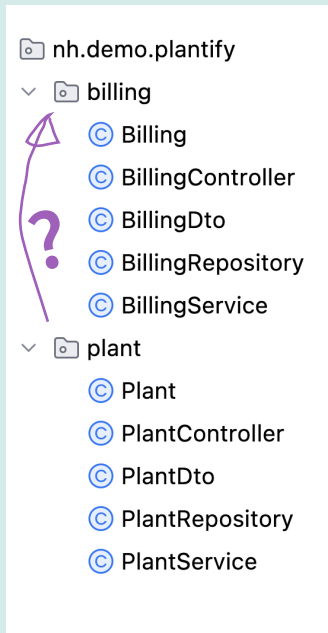
```
graph TD; nh_demo_plantify[nh.demo.plantify] --> billing[billing]; nh_demo_plantify --> plant[plant]; billing --> Billing[Billing]; billing --> BillingController[BillingController]; billing --> BillingDto[BillingDto]; billing --> BillingRepository[BillingRepository]; billing --> BillingService[BillingService]; plant --> Plant[Plant]; plant --> PlantController[PlantController]; plant --> PlantDto[PlantDto]; plant --> PlantRepository[PlantRepository]; plant --> PlantService[PlantService];
```

nh.demo.plantify

- ▼ billing
 - ⊙ Billing
 - ⊙ BillingController
 - ⊙ BillingDto
 - ⊙ BillingRepository
 - ⊙ BillingService
- ▼ plant
 - ⊙ Plant
 - ⊙ PlantController
 - ⊙ PlantDto
 - ⊙ PlantRepository
 - ⊙ PlantService

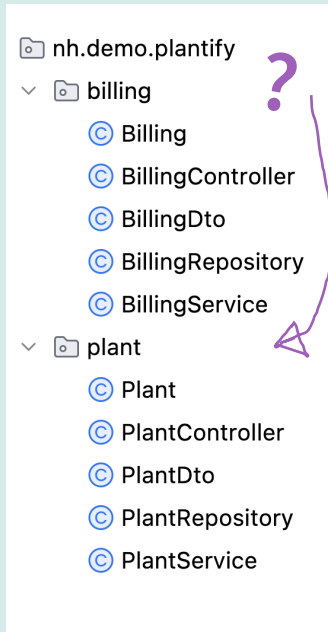
- Problem: Fachliche Packages existieren nicht isoliert
- Billing soll z.B. neue Pflanzen berechnen

- Gliederung der Anwendung in fachliche Packages



- Problem: Fachliche Packages existieren nicht isoliert
- Billing soll z.B. neue Pflanzen berechnen
- Wie sehen Abhängigkeiten aus?

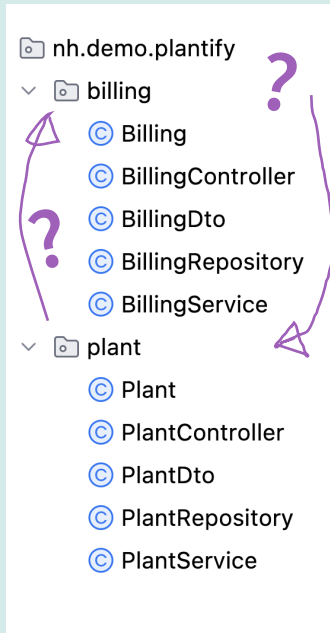
- Gliederung der Anwendung in fachliche Packages



- Problem: Fachliche Packages existieren nicht isoliert
- Billing soll z.B. neue Pflanzen berechnen
- Wie sehen Abhängigkeiten aus?
- Oder so?

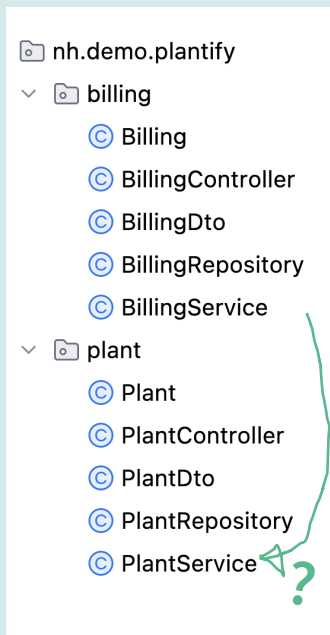
ARCHITEKTUR VON PLANTIFY

• Gliederung der Anwendung in fachliche Packages



- Problem: Fachliche Packages existieren nicht isoliert
- Billing soll z.B. neue Pflanzen berechnen
- Wie sehen Abhängigkeiten aus?
- Oder so?
- Mal so, mal so 🤔

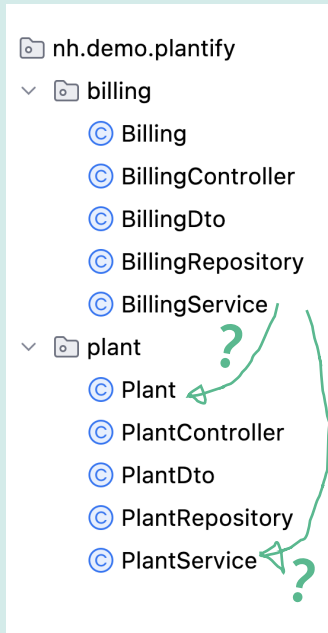
• Schnittstellen



- Problem: Fachliche Packages existieren nicht isoliert
- Billing soll z.B. neue Pflanzen berechnen
- Wie sieht die **Schnittstelle** von „plant“ aus?

ARCHITEKTUR VON PLANTIFY

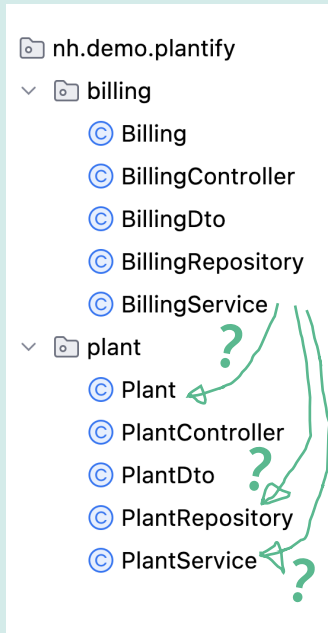
• Schnittstellen



- Problem: Fachliche Packages existieren nicht isoliert
- Billing soll z.B. neue Pflanzen berechnen
- Wie sieht die **Schnittstelle** von „plant“ aus?
- 🤔

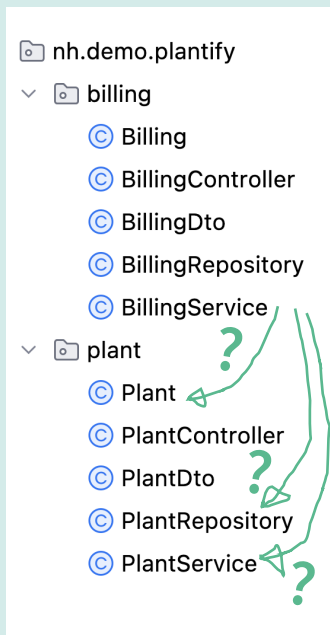
ARCHITEKTUR VON PLANTIFY

• Schnittstellen



- Problem: Fachliche Packages existieren nicht isoliert
- Billing soll z.B. neue Pflanzen berechnen
- Wie sieht die **Schnittstelle** von „plant“ aus?
- 🤔
- 🤔 🤔

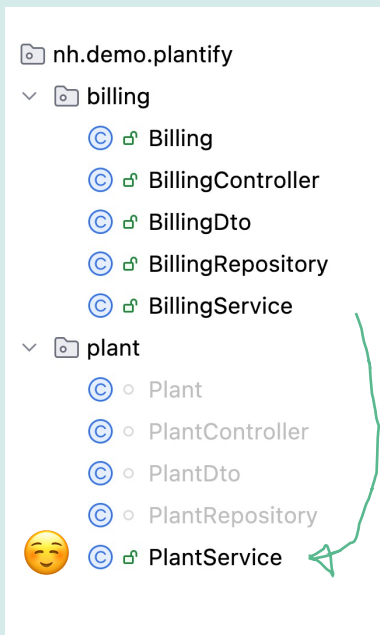
• Schnittstellen



- Problem: Fachliche Packages existieren nicht isoliert
- Billing soll z.B. neue Pflanzen berechnen
- Wie sieht die **Schnittstelle** von „plant“ aus?
- 🤔
- 🤔 🤔

ARCHITEKTUR VON PLANTIFY

- Schnittstellen per Java Bord-Mittel



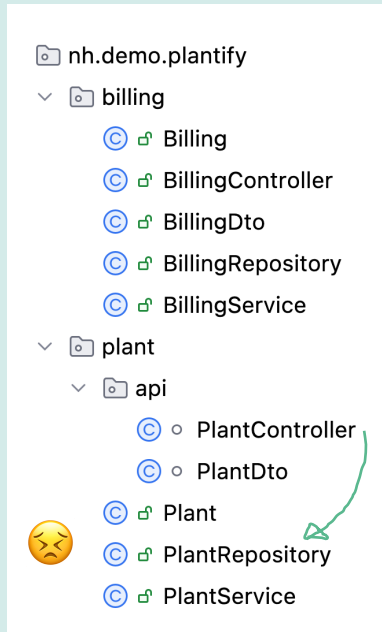
- Sichtbarkeit von Klassen einschränken

```
package nh.demo.plantify.plant;  
  
// Öffentlich  
public class PlantService {  
  
    // ...  
  
}
```

```
package nh.demo.plantify.plant;  
  
// Nur Plant-intern  
class PlantRepository {  
  
    // ...  
  
}
```

ARCHITEKTUR VON PLANTIFY

- Schnittstellen per Java Bord-Mittel



- Sichtbarkeit von Klassen einschränken

```
package nh.demo.plantify.plant;  
  
// Öffentlich  
public class PlantService {  
  
    // ...  
  
}
```



```
package nh.demo.plantify.plant;  
  
// Nur Plant intern  
public class PlantRepository {  
  
    // ...  
  
}
```

- Fachliche Packages

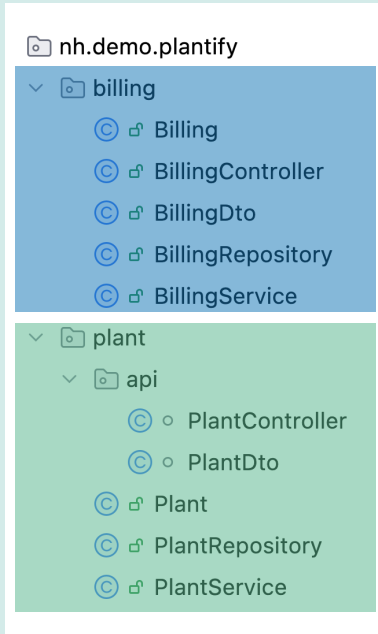
Was fehlt?

```
nh.demo.plantify
├── billing
│   ├── Billing
│   ├── BillingController
│   ├── BillingDto
│   ├── BillingRepository
│   └── BillingService
├── plant
│   └── api
│       ├── PlantController
│       ├── PlantDto
│       ├── Plant
│       ├── PlantRepository
│       └── PlantService
```

- Fachliche Packages

Was fehlt?

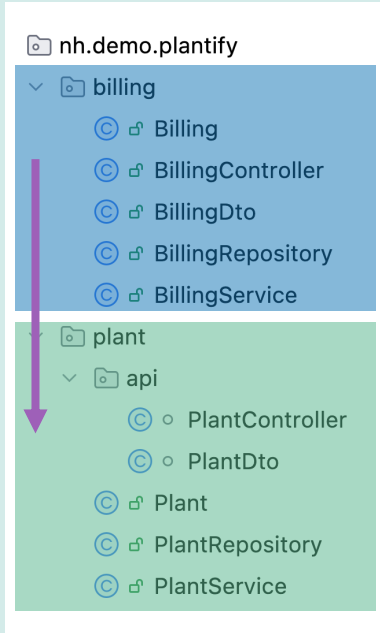
- „Echte“ Module
 - Mehr als Klassen oder Packages



- Fachliche Packages

Was fehlt?

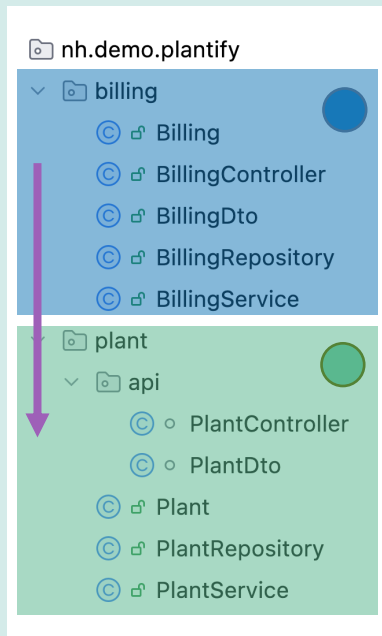
- „Echte“ Module
 - Mehr als Klassen oder Packages
- Klare Abhängigkeiten
 - Zwischen den Slices



- Fachliche Packages

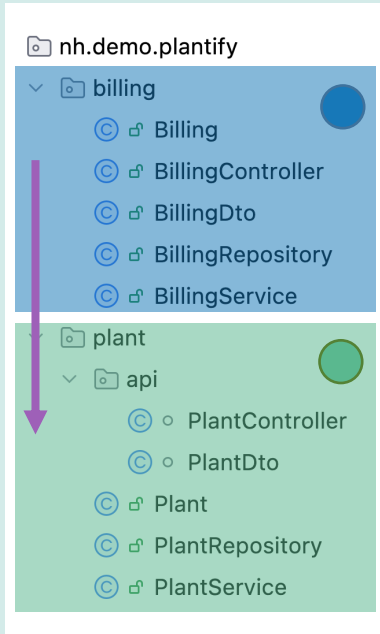
Was fehlt?

- „Echte“ Module
 - Mehr als Klassen oder Packages
- Klare Abhängigkeiten
 - Zwischen den Teilen
- Klare Schnittstellen
 - Zwischen den Teilen



SPRING MODULITH

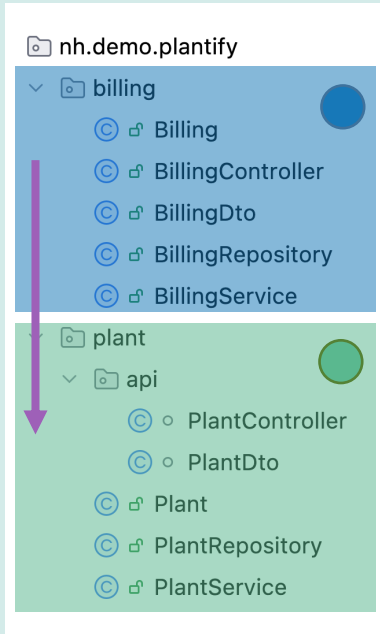
- Spring Modulith



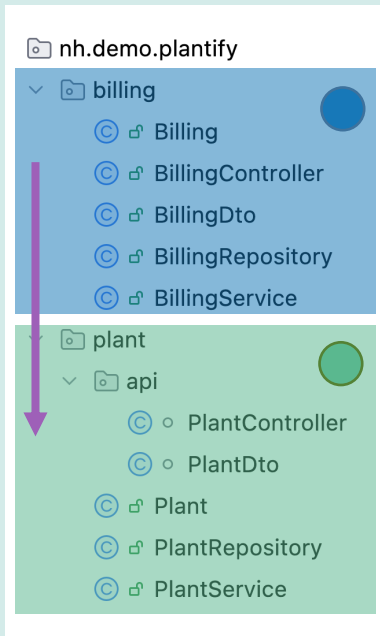
SPRING MODULITH

- Spring Modulith

- Packages werden zu ApplicationModules

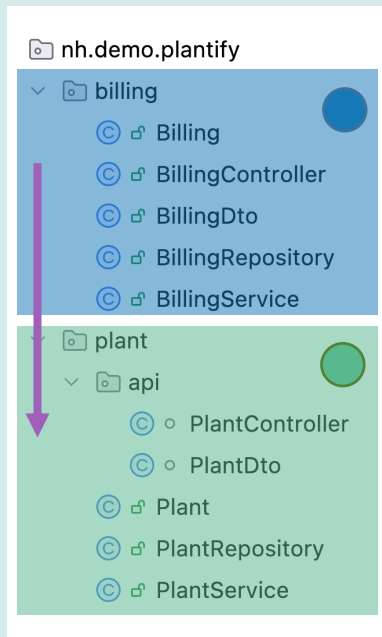


- Spring Modulith



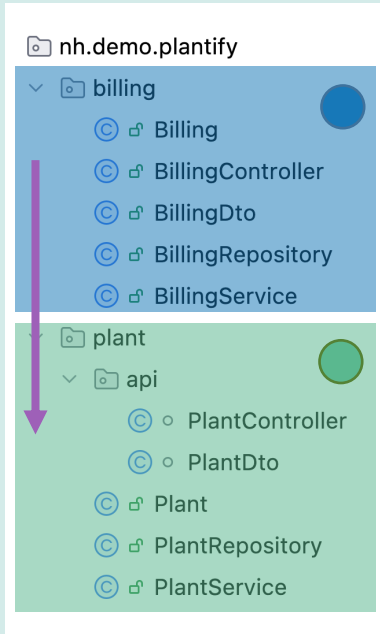
- Application Modules
 - Direkte Packages unterhalb des main-Packages
 - Definierte Abhängigkeiten
 - Explizite Schnittstellen

- Spring Modulith



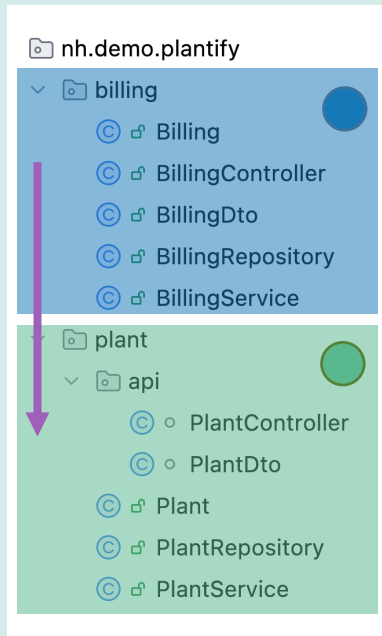
- Application Modules
 - Direkte Packages unterhalb des main-Packages
 - Definierte Abhängigkeiten
 - Explizite Schnittstellen
- Entkopplung über Events

- Spring Modulith



- Application Modules
 - Direkte Packages unterhalb des main-Packages
 - Definierte Abhängigkeiten
 - Explizite Schnittstellen
- Entkopplung über Events
- Isolierte Tests

- Spring Modulith



- Application Modules

- Per Default:  ges
-  keiten
-  Schnittstellen

- Entkopplung über Events

- Isolierte Tests

NILS HARTMANN

<https://nilshartmann.net>

Vielen Dank!

Slides & Code: <https://spring.schule/jax2026-spring-modulith>

Fragen und Kontakt:

nils@nilshartmann.net

<https://nilshartmann.net/kontakt>

